FPGA Partitioning That Respects Paths, Capacity and Connectivity What the Tool Does, What Improved, and What Was Verified

RapidWright partitioner validation report

20 October 2026; large-design measurements from 15–16 October

Abstract

This partitioner takes a synthesized design and a system topology, chooses which logic belongs on each FPGA, and writes `mapping.txt`. It tries to keep signals from crossing FPGA boundaries repeatedly before reaching a register, reduce communication, and keep every FPGA below its hard LUT limit. The user can give it more search attempts or allow uneven LUT use when that helps communication and paths. The backend is pinned to Mt-KaHyPar v1.6.2.

The evidence covers NVIDIA NVDLA, OpenTitan Earl Grey, a 3,072-core PicoRV32 mesh, and an AES farm, each on one through four J4 targets. The strongest observed improvements include reducing NVDLA's worst uncaptured path from three FPGA crossings to two, reducing OpenTitan's from four crossings to three in a two-FPGA run, and improving several mesh path and communication counts together. Other cases show compromises rather than universal improvement. More search sometimes found a better candidate and sometimes stopped improving.

The selected mappings have independent Vivado resource and boundary checks. A later mapping-format repair also checked all sixteen maps against the exact synthesized hierarchy, without changing functional ownership. These are source-netlist results. They do not establish placed-and-routed timing, actual pin multiplexing, bitstreams or achieved DUT clock frequency.

Contents

1 What a prototyping engineer can expect **2**

1.1 Capacity is a limit; balance is a preference 2

2 The numbers, in plain English **3**

2.1 What counts as a snake path? 3

2.2 I/O cuts, hypergraph cuts and pins are different counts 3

2.3 What does timing-aware mean here? 4

3 Does giving it more time produce a better partition? **5**

3.1 OpenTitan: improvement by attempt six, then no further accepted change 5

4 What improved on NVDLA, OpenTitan, mesh and AES? **7**

4.1 NVDLA: a clear four-FPGA improvement, with mixed smaller cases 7

4.2 OpenTitan: spending some communication to shorten bad paths 8

4.3 Mesh: less communication and less transit together . . . 8
4.4 AES: fitting a balance policy can cost communication or path quality . . . 9

5 The selected J4 results at a glance **9**

6 Reading a generated partition report **10**

7 The mapping-file contract is a hard requirement **13**
7.1 What the 20 October audit found and fixed . . . 13

8 Why these design choices have a research basis **14**
8.1 How topology awareness was checked . . . 14

9 How strong is the validation? **15**

10 What remains unproven **16**

A Complete selected-case counts **16**
A.1 Native LUT use by FPGA . . . 17
A.2 Exact crossing counts and categories . . . 17

B Technical definitions for developers **18**
B.1 Primitive behavior . . . 18
B.2 Source timing weights . . . 19
B.3 Capacity and balance arithmetic . . . 19

C Evidence files and what they establish **19**
C.1 The large mesh boundary-extraction repair . . . 20
C.2 Historical clean-clone and runtime checks . . . 20
C.3 Reproducing a result rather than just rerunning a command . . . 21
C.4 Key original identities . . . 21

D Historical mapping identities **22**

1 What a prototyping engineer can expect

The normal input is a post-synthesis checkpoint, a topology JSON file, and optionally an FPGA count. If the count is omitted, the tool uses all FPGAs described by the topology. Each requested FPGA must receive useful nonconstant logic. Java, Python, Vivado 2025.1 and Mt-KaHyPar v1.6.2 must be available through the documented setup.

A typical four-FPGA job is:

```
.\partitioner.cmd --input "C:\designs\synth.dcp" ‘
  --system-topology "C:\systems\machine.json" ‘
  --target-part VP1902 --fpga-count 4 --iterations 3 ‘
  --output "C:\results\four_fpga_run"
```

The part can be omitted when the topology or its supported companion already identifies the targets. The output directory must be new or empty. On a POSIX host, use `./partitioner` with that host's paths. These are command examples; the placeholder paths are not experiments.

The main deliverable is `mapping.txt`. It names exact synthesized instances and their logical FPGA owners. It does not alter the netlist, insert communication hardware, add pipeline registers, or run the downstream regroup tool.

The optimization priorities are practical:

1. Respect hard resource limits and populate every requested FPGA.
2. Reduce the worst paths that cross FPGA boundaries before reaching a register.
3. Use the available source timing sample to identify important paths.
4. Reduce the remaining long-crossing endpoints and combinational transit logic.
5. Reduce communication while accepting a sensible amount of uneven LUT use.

The exact acceptance order and guard limits are recorded in each run. A result can improve one quantity while worsening another; this report shows those cases explicitly.

1.1 Capacity is a limit; balance is a preference

The selected target part supplies the actual LUT capacity. The accepted mapping must fit the smaller of the requested explicit caps and 75% of that part's LUT capacity. For VP1902, the queried capacity is 8,460,288 LUT elements, so the 75% policy limit is 6,345,216 LUTs per FPGA. VU19P was separately checked at 4,085,760 elements, giving a 3,064,320 limit. Unsupported or conflicting part information is rejected rather than guessed.

The 75% ceiling is the user's engineering rule. It leaves room for implementation, but it is not a promise that any design below 75% will route. Wrapper, transport and debug logic are not included in these original-source subset measurements.

Balance asks how evenly the used LUTs are spread. For equal parts, 30% imbalance means that the busiest FPGA uses 30% more LUTs than the mean. It does not mean 30% device utilization, or a 30-percentage-point difference between two devices. With two FPGAs using one million and three million LUTs, the mean is two million and the reported imbalance is 50%. That can still be acceptable if both devices fit and the user permits that search envelope.

The preferred imbalance remains 20%, but the new default search allows much more uneven candidates when they fit the devices. It derives the automatic search envelope from the selected parts' physical capacities rather than imposing a 30% search envelope. For four equal parts its scalar envelope is 300%; for three it is 200%. These are permissive search bounds, not requested device utilizations, and useful logic on every FPGA is still mandatory. The hard Native LUT caps decide physical fit. This avoids chasing near-identical LUT counts simply for their own sake.

To request a tighter 70% search envelope, use `--search-balance0.70`. The separate raw-LUT guard is controlled by `--logical-balance`.

The preferred balance target remains soft. The historical experiments in this report used their recorded 28%, 30% or 70% envelopes. Their results have not been rerun or relabeled as measurements of the new defaults.

2 The numbers, in plain English

2.1 What counts as a snake path?

Consider a signal that travels from FPGA A through combinational logic on FPGA B and then leaves B without being captured by a register. That is the transit behavior this tool tries to reduce. It matters even when the next destination is C: A–B–C has no return to an earlier FPGA, but B still adds an uncaptured boundary transit. A–B–A is another bad case. Counting only returns to an earlier FPGA would miss the first example.

The report uses three separate path measurements:

Worst uncaptured crossings: The largest number of FPGA-boundary changes on a modeled path before the next register or terminal. Lower is better. A path staying on one FPGA has zero. A–B–C has two.

Extra crossings at endpoints: For each register input or modeled terminal, take its worst incoming uncaptured path, count crossings beyond the first, and add those excesses across endpoints. This shows how widespread multi-boundary paths are. It is an endpoint-based sum, not a count of every distinct path.

Combinational transit cells: The number of combinational primitives on an FPGA that can be reached from a cut input and can reach a cut output without a register in between. This counts cells, not paths. It shows how much logic participates in transit.

Older raw reports call these quantities `C` or `H`, `E`, and `V`. The tables below spell out their meaning. On J4, every selected pair is directly connected, so the worst minimum physical-link hop count equals the worst logical crossing count. That equality does not hold on every topology.

These measurements describe the supported structural dependency model. They do not prove that a path can be sensitized by real inputs, and they are not measured delays. Registers stop an uncaptured path; simply placing a register somewhere inside a large module does not make the whole module a register boundary.

2.2 I/O cuts, hypergraph cuts and pins are different counts

A net can have one driver and several destinations. The following example explains why a single number called "cuts" is ambiguous. If a driver on A sends one bit to both B and C, there is one crossing electrical

net, two directed destination deliveries, and one participating bit at each of three FPGA boundaries. A has two outgoing deliveries but can share the same logical source bit. The eventual transport may route or multiplex these deliveries differently.

Crossing electrical nets: Count each original nonconstant source net once when it crosses a partition boundary. Electrical aliases are joined before counting.

Directed destination deliveries: Count each crossing net once per remote destination FPGA. This is a useful logical communication measure, especially for fanout.

Per-FPGA I/O cut bits: The chart adds distinct outgoing and distinct incoming canonical data/functional-control net bits at that FPGA. Sending the same bit from A to both B and C counts once at A, once at B and once at C. Clock-only and reset-only nets are excluded from this chart and reported separately. Summing the chart values counts both ends of a point-to-point crossing.

Cut hypergraph edges: Count the abstract hyperedges whose vertices lie in more than one partition. A hyperedge is a solver representation, possibly after grouping hierarchy. Its cut objective is not automatically the same as the full electrical boundary census.

Optimization transfers: The solver/refiner's directed structural net-to-destination count. Some clock, reset and external-I/O scopes are excluded from this optimization view. Older raw reports call it `T`.

Neither crossing nets nor destination deliveries are physical pins, transport channels, TDM slots or a measured pin-multiplexing ratio. Those depend on the platform's links, routing, clocks, sharing rules and transport implementation. Ordinary clocks and reset-only distribution are reported separately from data and functional synchronous control. This separation matters because the platform may distribute clocks or resets outside its normal data pinmux.

The selected-map tables use independent Vivado boundary measurements where available. New run reports must state whether a number comes from the partitioner's source analysis or an independent Native Vivado boundary comparison. A Native LUT-resource check alone does not make the I/O counts independently verified.

2.3 What does timing-aware mean here?

The tool can export source setup paths and slack from Vivado and use them to weight communication and rank changes. Each selected multi-FPGA benchmark used a sample of 1,000 source setup paths. More critical sampled paths receive more attention. The tool also audits the worst modeled uncaptured crossings throughout its supported acyclic structural graph, beyond just the sampled timing paths.

This is timing guidance, not complete timing closure. The tool does not rerun a full platform timing analysis after each move. Many selected OpenTitan, NVDLA and mesh samples have zero modeled internal crossing cost; that does not mean zero delay, and those cases cannot demonstrate an improvement in sampled timing cost. AES does have nonzero sampled costs, with mixed results below. No measured inter-FPGA delay was supplied for the selected J4 studies, so this report makes no DUT-frequency claim.

3 Does giving it more time produce a better partition?

Yes, it can. It can also plateau. `-iterations` gives the frontend more feedback rounds using actual Vivado resource measurements. `-starts` gives the backend more starting assignments in each round. `-refine-passes` gives local path cleanup more passes. The default balanced effort is three rounds, three starts per round and three cleanup passes. Thorough effort uses five rounds, seven starts and six passes. Explicit flags override those presets.

A feedback round does more than repeat the same assignment: it can adjust proxy resource guidance after checking real Vivado LUT use. The tool keeps the best candidate it has accepted under its stated policy; rejected candidates do not replace it. More rounds therefore create more opportunities, but do not guarantee improvement in every measurement. The multithreaded backend also need not produce identical answers from a seed alone.

The revised frontend runs the requested multi-FPGA feedback rounds by default, even when it has already removed repeated uncaptured crossings. That leaves room to improve the remaining communication. Optional `-early-stop` restores the earlier stopping rule: stop when a fully analyzed, within-envelope candidate has at most one uncaptured crossing, no combinational transit cells and no extra topology-hop requirement. That rule does not establish that all I/O cuts are minimal. A one-FPGA request needs only its direct single assignment and no external partitioning backend.

3.1 OpenTitan: improvement by attempt six, then no further accepted change

A four-FPGA OpenTitan experiment allowed 70% imbalance and ran twelve feedback rounds with twelve starts each: 144 starting solutions. The selected result was found on attempt six. Its worst uncaptured path improved from five crossings to four. Combinational transit cells fell from 6,842 to 5,469, and optimization transfers fell slightly from 3,481 to 3,476. The endpoint-excess sum became worse, which the higher-priority worst-path improvement allowed.

Measurement	First accepted	Selected, attempt 6
Worst uncaptured crossings	5	4
Extra crossings at endpoints	23,786	41,688
Combinational transit cells	6,842	5,469
Optimization transfers	3,481	3,476
Largest LUT use above the mean	16.27%	3.42%

Attempts two through five and seven through twelve were rejected by the retained quality guards. No new accepted improvement appeared after attempt six. Both accepted candidates had actual Native LUT measurements. The final selected map also passed an independent full-source boundary/category comparison: 3,402 data/functional-control crossing nets and 3,594 directed data/control destination deliveries.

This longer run did not beat every earlier result. The earlier qualified four-FPGA map also had four worst crossings, with a smaller endpoint-excess sum of 27,667 and 3,529 optimization transfers. Its transit-cell count was 6,002. The newer run traded fewer transit cells and transfers for many more extra crossings at endpoints. Under the recorded selection order, the earlier result remained preferable. The original J4 package was therefore retained.

Two additional profiles raised the total to 23 feedback attempts and 276 starting solutions. A coarser grouping with a 15% communication-growth allowance eventually selected a map at 48.48% imbalance, 20,323 endpoint excess, 2,404 transit cells and 4,090 transfers. However, its worst path had five crossings

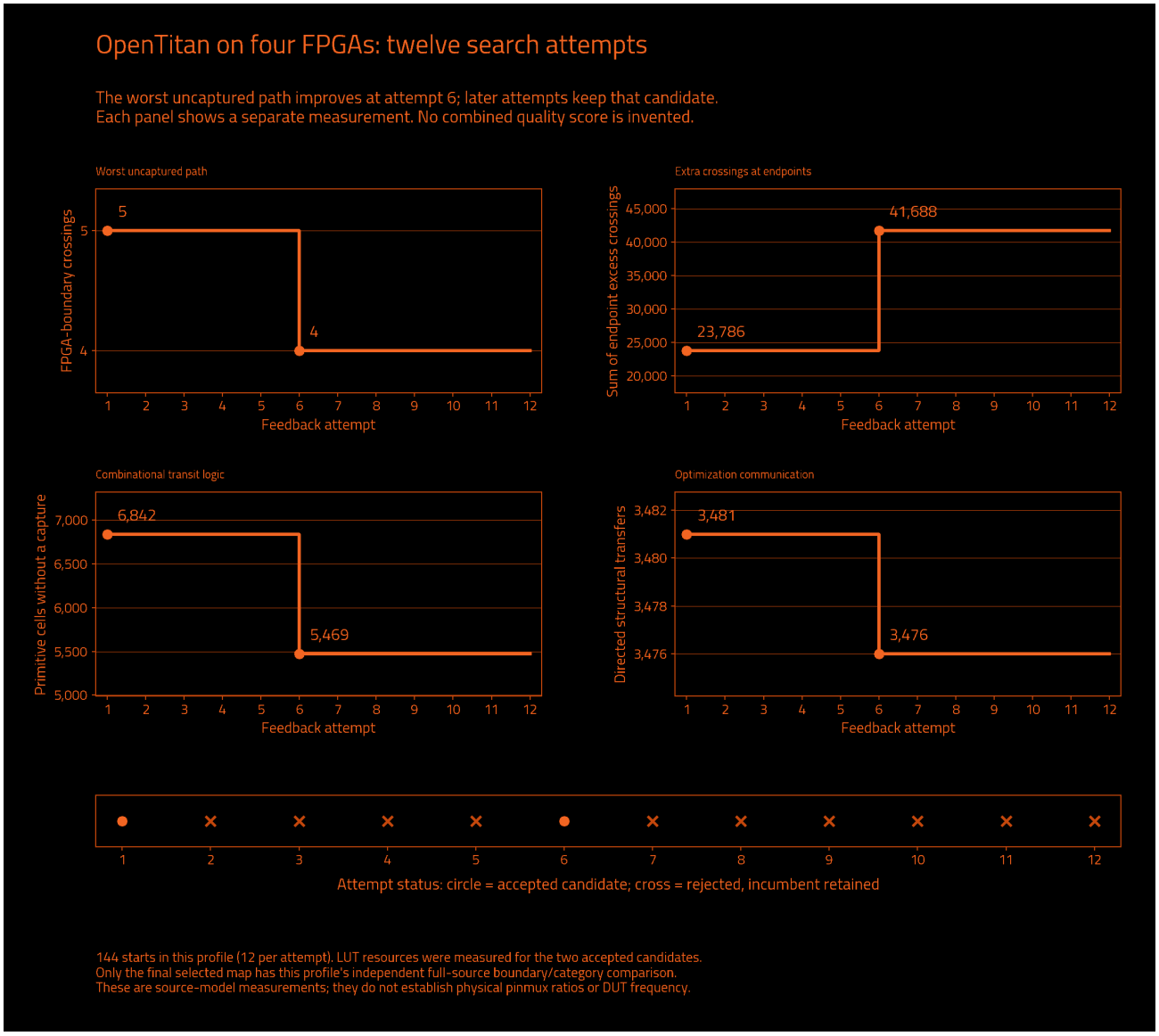


Figure 1: OpenTitan's twelve-attempt search. The best retained candidate improves at attempt six; additional attempts do not produce another accepted change. Rejections are not zero-cost candidates. These are measurements within this run, not comparisons of different FPGA counts.

rather than four. A separate three-round profile with a 20% allowance retained four worst crossings, 27,768 endpoint excess, 7,833 transit cells and 3,679 transfers. Settings changed across these profiles, so they are alternatives, not a controlled proof that one switch caused an improvement. All three selected profile results received full-source boundary/category comparisons. Those proofs do not apply to every intermediate candidate.

The practical lesson is to allow longer search when the run is still improving, then inspect the actual tradeoffs. A wide imbalance envelope permits uneven maps; it does not force one. It also does not guarantee fewer I/O cuts or shorter paths.

4 What improved on NVDLA, OpenTitan, mesh and AES?

The following before/after comparisons come from recorded candidate traces. "Before" is the actual initial assignment for that candidate. "After" is the retained assignment, later checked for Native resources and selected-map boundaries. Initial assignments did not all receive those later independent checks. The comparisons show what the algorithm changed, without pretending that every earlier candidate was a released result.

4.1 NVDLA: a clear four-FPGA improvement, with mixed smaller cases

Four-FPGA NVDLA measurement	Before	After
Worst uncaptured crossings	3	2
Extra crossings at endpoints	1,001	733
Combinational transit cells	266	230
Optimization transfers	8,093	8,110

The worst uncaptured path became shorter by one crossing. Endpoint excess fell 26.77% and transit cells fell 13.53%, for only 17 additional optimization transfers, a 0.21% increase. Further swaps improved the first two-crossing result from 772 endpoint excess and 248 transit cells to 733 and 230. A later proposal that returned to three worst crossings was rejected.

The final Native LUT counts were 299,097 / 490,065 / 367,183 / 547,391, giving 28.52% imbalance. The independent final boundary check matched 8,313 crossing electrical nets and 8,320 directed destination deliveries, including clocks and resets. These counts differ from the 8,110 optimization transfers because their policies differ.

The two-FPGA case is less favorable: endpoint excess fell from 4,091 to 1,886, but transit cells rose from 19 to 1,972 and transfers rose from 2,942 to 3,236, with no change in the three-crossing worst path. The ranked policy gives endpoint excess priority over transit cells; it therefore permits this compromise. It would be inaccurate to call it an improvement in every snake-path measure. A componentwise guard is available when each listed path measurement must avoid regression.

The three-FPGA result also shows a small compromise: endpoint excess fell from 38,568 to 37,521 while transit cells rose from 3,716 to 3,717 and transfers rose from 5,768 to 6,026. These cases show why a readable report needs several path and communication measures.

4.2 OpenTitan: spending some communication to shorten bad paths

Two-FPGA OpenTitan measurement	Before	After
Worst uncaptured crossings	4	3
Extra crossings at endpoints	18,173	7,950
Combinational transit cells	1,125	1,119
Optimization transfers	1,732	1,869

This is a useful example of the user's preferred tradeoff: a shorter worst path and much less endpoint excess, at the cost of more communication. Transfers grew 7.91% from this candidate's initial assignment. The run's original fixed reference was 1,804 transfers, making the retained value 3.60% above that reference and inside its 10% allowance.

The three-FPGA result shortened the worst path from five crossings to four, reduced endpoint excess from 37,872 to 36,693, and reduced transit cells from 3,851 to 2,966. Transfers rose from 2,755 to 3,258. That is an 18.26% change relative to this candidate's start, but only 4.52% above the run's original reference of 3,117. Budgets remain tied to the original reference; they do not compound after each accepted pass.

The four-FPGA candidate shortened its worst path from five crossings to four and reduced endpoint excess from 43,567 to 27,667. Transit cells rose from 5,136 to 6,002 and transfers rose from 3,375 to 3,529. This is another deliberate ranked compromise, not an across-the-board improvement.

The three-FPGA feedback history gives a concrete reason to allow uneven LUT use. Its first resource-checked candidate had only 6.72% imbalance, but five worst crossings, 5,165 transit cells and 3,338 transfers. The selected later candidate had 28.44% imbalance, four worst crossings, 2,966 transit cells and 3,258 transfers. It used balance less evenly while improving those three measurements. Endpoint excess did become worse, from 27,724 to 36,693. Seeds and feedback changed between candidates, so this shows a useful measured compromise rather than proving that relaxing balance alone caused it.

4.3 Mesh: less communication and less transit together

Two three-FPGA mesh feedback candidates used the same source, topology and model. Both had actual Native resource measurements. The selected second candidate then received the independent final boundary proof.

Three-FPGA mesh measurement	Candidate 1	Candidate 2
Worst uncaptured crossings	2	2
Extra crossings at endpoints	66	32
Combinational transit cells	813	720
Optimization transfers	6,988	6,452
Largest LUT use above the mean	25.0109%	25.0140%

Transfers fell 7.67%, endpoint excess fell 51.52%, and transit cells fell 11.44%, while worst crossings stayed at two. The imbalance change was only 0.0031 percentage points. Within the selected candidate, one accepted local move reduced transit cells from 1,296 to 720 while leaving worst crossings, endpoint excess and transfers unchanged. A later proposal with three worst crossings was rejected.

Seeds and feedback guidance evolved between candidates, so this is evidence that the search found a better measured tradeoff; it does not isolate one mechanism as the cause. The sampled source timing cost was zero in both candidates, so this example does not demonstrate an improved timing score.

4.4 AES: fitting a balance policy can cost communication or path quality

AES follow-up runs produced maps within the recorded 30% imbalance envelope. The earlier main alternatives exceeded that envelope. The accepted follow-ups also met tighter experimental per-device LUT caps of 153,881 / 102,587 / 76,940 for two / three / four FPGAs.

FPGAs	Candidate	Imbalance	Worst crossings	Transfers
2	Earlier, outside envelope	30.60%	2	8,172
2	Selected follow-up	25.62%	3	8,440
3	Earlier, outside envelope	32.85%	3	13,804
3	Selected follow-up	27.26%	3	14,133
4	Earlier, outside envelope	30.79%	3	15,997
4	Selected follow-up	26.98%	3	16,800

For two FPGAs, the follow-up improved balance, endpoint excess and sampled timing cost, but worsened the worst crossing count, transit cells and transfers. For three, balance improved while sampled timing cost and all three secondary path/communication measures became worse. For four, balance, endpoint excess and transit improved, while sampled timing cost and transfers became worse.

These observations support allowing sensible imbalance when the devices fit. They do not justify changing the selection policy after seeing a favorable-looking metric. The historical selected maps remain the results of their stated 30% envelope. A two-FPGA alternative with 28.9985% imbalance had fewer transit cells and transfers than the selected follow-up, but worse endpoint excess and sampled timing cost; neither wins every measure. A separate coarse case remained outside the envelope and lacked a completed exact-map category comparison, so it was not promoted to the selected package.

More effort, grouping and budget settings differed across these AES follow-ups. This was engineering tuning, not an isolated test proving that one flag caused a particular change.

5 The selected J4 results at a glance

All selected targets are VP1902. Every row below passed its effective hard Native LUT caps and contains useful logic on every requested FPGA. A one-FPGA map naturally has no inter-FPGA cuts; it checks assignment and accounting rather than boundary quality. The full resource and cut-category tables are in Appendix A.

Design	FPGAs	Worst uncaptured crossings	Extra crossings at endpoints	Combinational transit cells
OpenTitan Earl Grey	1	0	0	0
OpenTitan Earl Grey	2	3	7,950	1,119
OpenTitan Earl Grey	3	4	36,693	2,966
OpenTitan Earl Grey	4	4	27,667	6,002
NVDLA	1	0	0	0
NVDLA	2	3	1,886	1,972
NVDLA	3	5	37,521	3,717
NVDLA	4	2	733	230
PicoRV32 mesh	1	0	0	0
PicoRV32 mesh	2	1	0	0
PicoRV32 mesh	3	2	32	720

Design	FPGAs	Worst uncaptured crossings	Extra crossings at endpoints	Combinational transit cells
PicoRV32 mesh	4	2	32	1,440
AES farm	1	0	0	0
AES farm	2	3	27,942	9,205
AES farm	3	3	30,604	7,723
AES farm	4	3	31,342	6,193

The number of FPGAs changes the partitioning problem; these rows are not a controlled before/after experiment. For example, NVDLA's four-FPGA result has better worst crossings than its three-FPGA result, but that does not prove that adding an FPGA always helps paths. Different search histories can find different local solutions.

The 3,072-core mesh is the largest source, with 11.22 million logical primitive leaves and 6,314,270 whole-source Native LUTs. Its one-FPGA result is only 30,946 LUTs below the VP1902 policy cap, before adding any downstream transport or debug logic. This small margin should be visible when deciding whether to implement that assignment.

6 Reading a generated partition report

Open `report.html` in the accepted output directory. It contains the selected mapping's measurements, figures, search history and validation limits. The three PNG files are `lut_balance.png`, `io_cut_balance.png` and `fpga_connectivity.png`. The corresponding numeric data and file identities are in `report_data.json` and `report_manifest.json`.

The accepted mapping is the selected candidate, not necessarily the last candidate attempted. Its report should be read in this order:

1. Check the acceptance status, selected FPGA count and target parts. Confirm that every requested FPGA has useful logic and passes its hard limit.
2. Read the LUT chart. The horizontal line is the mean; the deviation panel shows which devices are above or below it. A positive deviation is not a capacity violation unless the actual device limit is exceeded.
3. Read the I/O cut chart and its definition. Data/functional-control communication is separated from clock and reset-only distribution. A logical I/O cut count describes traffic to transport, not the final physical pinmux ratio.
4. Inspect the connectivity picture to find heavy communicating FPGA pairs. A dense logical relation does not itself choose a routed physical path through the topology.
5. Read worst uncaptured crossings, endpoint excess and transit cells together. A small total cut count can still hide one severe path.
6. If the search used several rounds, compare the accepted candidates and rejected attempts. A rejected attempt is not a new published mapping, and a missing measurement is not zero.

The plots describe the selected mapping. The iteration history explains how the search reached it. Complete raw reports and manifests remain useful for developers, while the HTML report is the human-readable entry point. A separate independent full boundary check, when present, must be identified explicitly; normal frontend success is not a claim that the benchmark's complete Native census was rerun.

The LUT chart uses Native Vivado CLB LUT counts for the original mapped source subsets. Its second panel shows signed percent deviation from the arithmetic mean. The I/O chart uses the partitioner's canonical source-netlist analysis, with the data/control policy above; it is not automatically a new independent Native boundary comparison. Its second panel uses the same deviation formula. When every value is zero, deviation is defined as zero instead of dividing by zero.

The connectivity figure counts data/control net bits for each directed FPGA pair. The A-to-B-and-C example contributes one delivery to A–B and one to A–C, even though A's outgoing chart count is only one bit. Small systems include a connection drawing and a count matrix; larger systems use the matrix to keep the result readable. These are logical deliveries awaiting transport design, not selected physical routes.

The HTML uses restrained black-and-white styling. PNGs use the requested orange/black palette and a bundled regular display font, so the installed system font list does not change their layout. The drawing dependencies are pinned in `scripts/partition/requirements.txt`; font and rendering sources are included with the package.

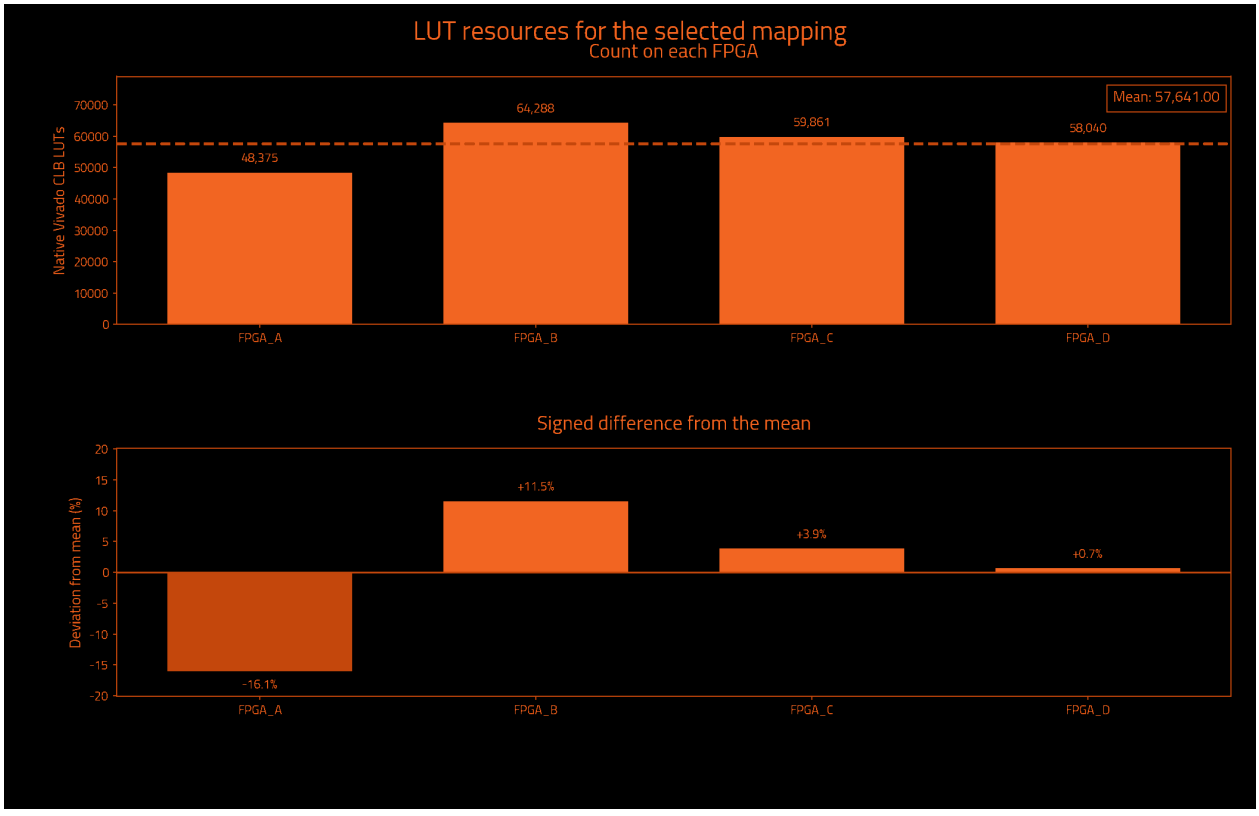


Figure 2: Example selected-map LUT report for four-FPGA OpenTitan. The top panel shows Native source-subset CLB LUTs and their mean. The bottom panel shows signed deviation from that mean, separate from device utilization. These historical Native measurements are preserved by the exact-ownership check; the repaired mapping bytes were not measured in a fresh Vivado run.

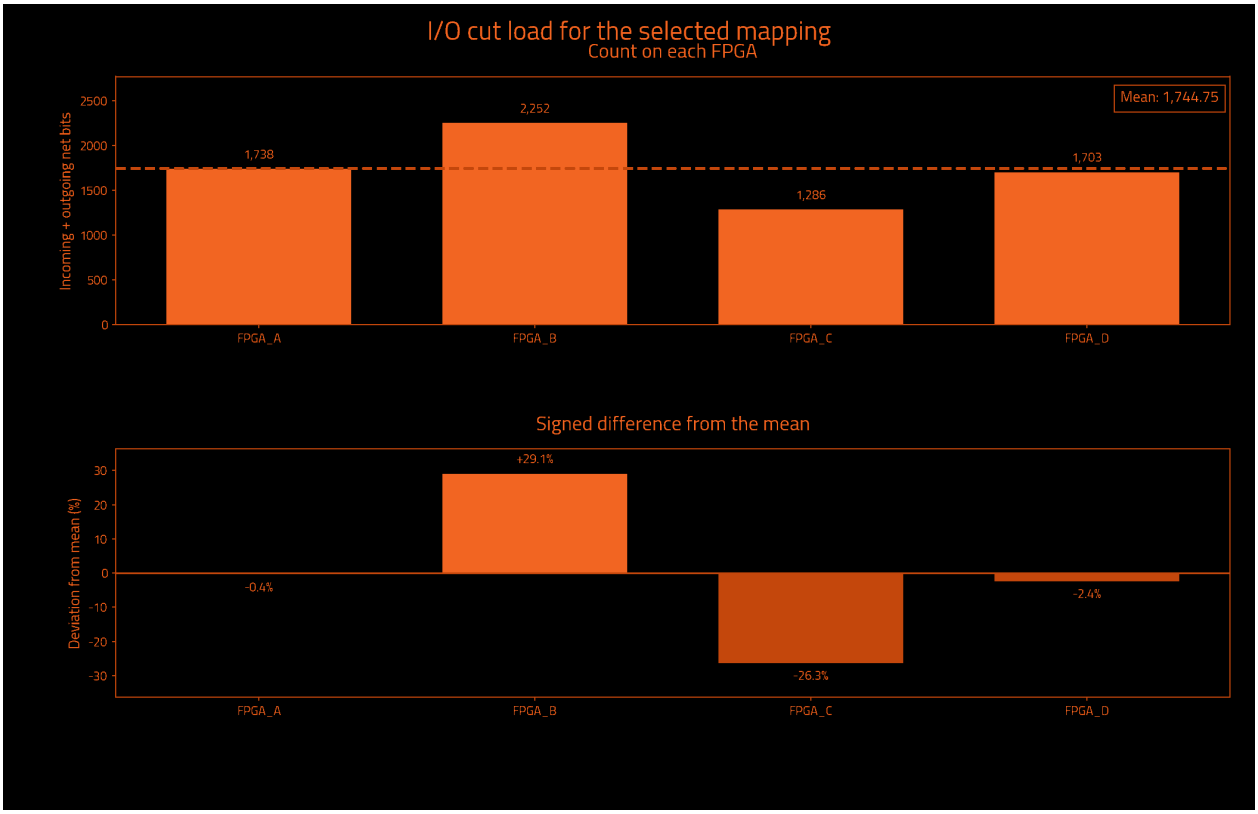


Figure 3: Example I/O communication report for the same selected map. Counts are unique outgoing plus incoming data/functional-control net bits per FPGA, with clock-only and reset-only distribution excluded. They are logical transport demand, not a physical pin count.

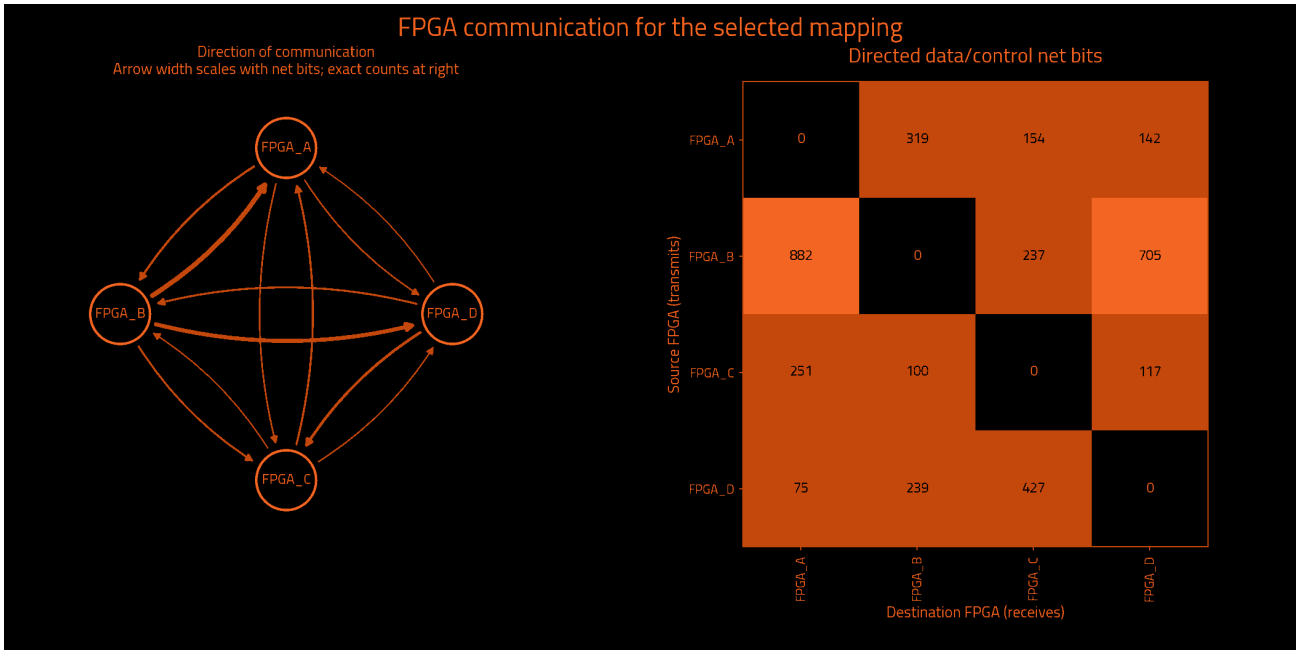


Figure 4: Example directed communication between FPGA pairs. Fanout creates one delivery for each remote FPGA. A declared physical topology and a logical communication pattern answer different questions; transport routing remains downstream.

7 The mapping-file contract is a hard requirement

The mapping consumer processes records in file order. It is therefore not enough to produce the right leaf-owner counts. The instance paths must exist in the exact synthesized design, and the move order must be valid.

A valid assignment line has exactly two whitespace-separated fields: the exact synthesized DUT instance path and a logical FPGA name such as `FPGA_1` or `FPGA_A`. Physical socket names belong to later board assignment. Generated files contain assignment lines only; the validator accepts blank lines and whole-line comments on input but rejects inline comments, duplicates, wildcards, quoted paths and one-field shortcuts.

The simplest two-partition example is:

```
hw_top/partition_b FPGA_2
hw_top FPGA_1
```

The first line extracts the child. The second moves the root's remaining logic to FPGA_1. There is no implicit rule assigning unlisted logic to FPGA_1: the explicit root line establishes that ownership.

Every mapped child must appear before every mapped ancestor. The root is last. Omit a child assignment when it has the same destination as its nearest mapped ancestor, because it adds no ownership information and can trigger downstream same-destination refactoring problems.

A nested return to the root FPGA is different and must remain:

```
hw_top/cluster_b/local_helper FPGA_1
hw_top/cluster_b/compute FPGA_3
hw_top/cluster_b FPGA_2
hw_top FPGA_1
```

Here the helper needs its own line: its nearest mapped ancestor, cluster_b, belongs to FPGA_2. Removing the helper just because it matches the distant root would assign it incorrectly. Sorting these records alphabetically or putting the root first would change their meaning.

Exact synthesized ancestry also matters. A slash can be part of an instance's local synthesized name. Splitting a string at every slash can invent a parent that does not exist. The revised writer follows actual EDIF parent objects and then checks how the serialized paths resolve. Generate indices, dots and selected branches must match the actual synthesized input. Explicit GND/VCC assignments are omitted by primitive type; constants and empty hierarchy cannot satisfy occupancy.

7.1 What the 20 October audit found and fixed

An exact-source audit reread all four source EDIFs and all sixteen delivered mappings. Twelve historical maps were valid; OpenTitan's two-, three- and four-FPGA maps and NVDLA's four-FPGA map contained nonexistent compressed parent prefixes. The earlier ownership/cut comparisons had resolved leaves correctly by string prefixes, but that did not prove the move consumer could find those parents.

The writer was repaired to use real parents. The successor package was then checked against every functional leaf of the original assignment: 59,952,616 ownership comparisons across the sixteen cases, with zero differences. Root/top-port ownership was also unchanged. All sixteen successor maps pass the exact-source contract. The 632 original package artifacts were preserved with unchanged hashes.

Use `RESULTS FOR J4/CONTRACT_COMPLIANT_20261020` for these revised move files. The original Native measurements still describe the same functional subsets and connections, but their receipts retain their original mapping hashes. They were not rewritten to pretend that Vivado had been rerun on the new file bytes.

The producer also emits record ancestry, intended leaf ownership and original source-connection manifests. Before final publication, the frontend checks the complete compressed streams, source identities, exact records, coverage and agreement with `cells.txt`. Compact and verbose modes obey the same contract. Source-free or unchecked writers refuse to publish a purportedly validated mapping.

This is a pre-regroup check. The unavailable downstream regroup/wrapper tool was not run. Promoted attributes, final clocks/resets, transport insertion and hardware behavior require later verification in that flow.

8 Why these design choices have a research basis

Mt-KaHyPar provides a mature multilevel hypergraph partitioning backend, including coarsening and local improvement [5]. Version 1.6.2 supports direct target-graph mapping with a Steiner-tree objective [3, 6]. This lets communication cost depend on the declared system connections during optimization, rather than merely renaming partitions afterward. The same version does not provide multidimensional resource constraints, so the surrounding flow checks resource vectors and asks Vivado to verify actual LUT use [4].

OpenROAD's TritonPart documentation describes using critical paths and slack during partitioning, then updating timing costs during refinement [1]. The related paper explains why initially noncritical paths also matter [2]. Those ideas motivated timing-guided weights and path-based cleanup here. This implementation is narrower than TritonPart's full timing updates: it re-evaluates the exported source sample and uses its own conservative register-boundary graph. It does not claim to reproduce the complete TritonPart timing engine.

Research on multi-FPGA TDM explicitly links routing, communication pressure and multiplexing delay [7, 8]. This supports reporting heavily loaded pairs and data/control categories. It does not supply a universal conversion from total cuts to DUT clock. EasyPart also treats FPGA resources and system topology as part of partitioning [9]. Public vendor material describes connectivity-aware, constraint-driven partitioning and timing-driven synthesis [10]; it does not disclose enough detail to reproduce a proprietary tool's algorithm or claim equal results.

No cited research establishes one universal 3%, 10%, 20% or 30% cut/balance allowance. The default 10% transfer-growth allowance is an explicit engineering choice reflecting the user's willingness to spend communication on severe path improvements. A different platform or design may justify another value. The hard device caps remain independent of those tradeoffs.

8.1 How topology awareness was checked

The code uses the selected target graph during backend initialization and local refinement. Path analysis also counts the minimum physical links needed between consecutive owners. Tests cover sparse connections, unreachable pairs, relay hops and the special case of a fully connected system. A relay hop is a lower bound based on declared connectivity; it is not a committed transport route or measured cable delay.

The sixteen selected large-design maps use J4, where selected FPGA pairs are directly connected. They

therefore exercise the real topology input and device selection, but do not demonstrate a large sparse-machine speedup. Version 1.6.2 supports a target graph of up to 64 nodes in this direct mapping API, which permits 24/28-node experiments. That capability is not evidence that a completed final 24/28-FPGA result exists in this report.

9 How strong is the validation?

There are three different kinds of evidence, and they should stay separate.

1. Source model and mapping tests

Tests cover registers as path barriers, combinational transit, returns to an earlier FPGA, memory/DSP stage settings, synchronous versus asynchronous controls, unknown primitive behavior, cycles, topology hops, timing weights, fixed assignments, useful occupancy, hard caps and retained/rejected moves. Mapping tests cover exact source ancestry, record order, nested exceptions, redundant records, malformed records, constants, full ownership coverage and publication guards.

2. Actual Vivado resource and boundary comparisons

Every selected large-design map has actual Native Vivado resource counts for its original-source logic subsets. Selected AES and OpenTitan maps also have comparisons against complete source-net censuses. NVDLA and mesh use exact crossings for their four selected maps, complete logical-owner/control metadata and top-port coverage; they do not dump every internal noncrossing net. All sixteen selected cases recorded zero missing, extra, endpoint, pair, primitive-histogram or category mismatches within those stated scopes.

Normal frontend success checks its model, capacities, useful occupancy, source identity and search envelope. It does not automatically repeat the separate complete benchmark boundary proof. A resource-only success must not be labeled as an independent full I/O-cut comparison.

3. Command and package integration

The 20 October mapping-contract revision passed 147 focused Java tests on Windows and Linux. The Python partition suite completed 187 tests per platform with zero failures and two optional recorded-Native integrations skipped. Seventeen native-reference helper tests passed on Windows; the optional WSL helper run lacked Tcl/Tk and is not called a Linux helper-suite pass.

A real small Vivado 2025.1 design also completed one-, two-, three-, default-four-FPGA and verbose-two-FPGA runs with the contract revision. Native LUT counts were 304; 185/119; 117/102/85; and 85/83/68/68, respectively. Every successful map covered 624 functional leaves and separately classified 32 constants. The connection manifest held 1,074 nets, 1,618 aliases and 3,378 endpoint bits. A one-LUT cap was correctly rejected without an accepted mapping. These are real integration checks, not sixteen new large-design optimization runs.

4. The reporting and capacity-first update

The revised Python suite ran 228 tests on both Windows and Linux, with zero failures and two expected skips for optional recorded-Native fixtures. All 17 source-reference helper tests also passed on each platform. The suite includes actual PNG rendering, exact palette checks, multicast counting, zero-count charts, 24/28-FPGA display cases, unsupported optional resource fields, mapping-contract cases and failures during final publication. Display tests for 24/28 FPGAs do not establish completed large-design partitions at those sizes.

A fresh small-design run used the new four-FPGA defaults with three requested attempts, one start per

attempt and source timing disabled. It tried all three attempts and retained the first accepted result after the later proposals failed its quality anchor. Vivado measured 165, 1, 137 and 1 LUTs on logical FPGAs A through D. Every FPGA contained useful logic; all 624 functional leaves retained an owner; each device remained below its hard cap. The 117.1% LUT imbalance exceeded the soft 20% preference and was accepted by the permissive default search. This checks that the new default does not silently force equal area. It is not evidence of a new improvement on NVDLA or OpenTitan.

That run published the selected mapping, three PNGs, HTML, JSON data and a hash manifest together. Its LUT chart values matched the finalized Vivado table exactly. An initial report-format failure exposed Vivado's literal `NA` for unsupported UltraRAM; the run withheld the accepted mapping. The fix keeps optional unavailable resources as unavailable rather than inventing a zero, while mandatory LUT counts remain strict. Browser appearance was not independently previewed: the available browser interfaces were unavailable or the preview action was blocked. The PNGs and compiled PDF were inspected directly.

The sixteen archived J4 examples include the new reports, exact mapping bytes and small proof summaries. Their large-design measurements remain historical values preserved by the exact ownership checks; they are not sixteen fresh optimization runs. The versioned `presentation_validation.json` records the current validation scope. The bundled Titillium Web Regular font is an openly licensed visual approximation for the requested commercial Klavika font; no machine font installation is needed.

10 What remains unproven

The tool is a heuristic. It can fail to find an acceptable result, and it does not prove global optimality. A hard-cap promise means that accepted outputs must satisfy the measured cap; it does not mean that every request has a feasible answer.

The structural path audit is complete only on the supported acyclic dependency model. Unknown behavior or cycles must be reported as incomplete, rather than silently treated as safe. The source STA sample does not establish timing for every path. No selected experiment models measured board transport delay, performs downstream wrapping, produces final routed target utilization, or measures hardware DUT frequency.

The next useful evaluation would hold the source, topology, grouping, budgets and effort fixed, compare several deterministic seeds across balance envelopes, and retain the nondominated alternatives. A sparse 24/28-FPGA study should report pair/link bottlenecks, relay-hop bounds and actual target limits. A DUT-frequency claim then requires platform transport/TDM timing and constrained routed timing or hardware measurements.

A Complete selected-case counts

The source configurations are the preserved synthesized experiments, not every possible configuration of their upstream projects. A source synthesized for a different part is not automatically retargeted, placed or routed onto VP1902 merely because VP1902 supplies the capacity policy.

Source	Logical primitive leaves	Whole-source Native LUTs	Native flip-flops
OpenTitan Earl Grey	434,666	230,543	146,801
NVDLA	2,869,930	1,703,727	764,988
PicoRV32 mesh	11,216,577	6,314,270	3,789,664
AES farm	517,544	236,741	232,541

Logical LUT primitives differ from Native CLB LUT use. The mesh has 7,105,564 logical LUT primitives, for example. LUT combining and memory packing are why original-source subset reports use Native Vivado as the resource authority. Subset totals can differ from the whole design when hypothetical partitioning removes cross-boundary combining opportunities.

A.1 Native LUT use by FPGA

A dash means that FPGA is not selected. The recorded packages use logical labels `FPGA_A` through `FPGA_D`. Their topology selections and physical-device associations are preserved in `fpga_devices.tsv`.

Design	FPGAs	FPGA A	FPGA B	FPGA C	FPGA D	Imbalance
OpenTitan	1	230,543	–	–	–	0.00%
OpenTitan	2	116,486	114,070	–	–	1.05%
OpenTitan	3	98,750	61,885	70,025	–	28.44%
OpenTitan	4	48,375	64,288	59,861	58,040	11.53%
NVDLA	1	1,703,727	–	–	–	0.00%
NVDLA	2	917,761	785,966	–	–	7.74%
NVDLA	3	511,063	641,556	551,108	–	12.97%
NVDLA	4	299,097	490,065	367,183	547,391	28.52%
Mesh	1	6,314,270	–	–	–	0.00%
Mesh	2	3,156,885	3,157,385	–	–	0.01%
Mesh	3	2,631,240	1,578,448	2,104,582	–	25.01%
Mesh	4	1,578,440	1,578,950	1,578,441	1,578,439	0.02%
AES	1	236,741	–	–	–	0.00%
AES	2	148,726	88,064	–	–	25.62%
AES	3	100,452	82,724	53,630	–	27.26%
AES	4	75,196	60,621	72,146	28,921	26.98%

A.2 Exact crossing counts and categories

OpenTitan and AES use full Native source-net censuses. NVDLA and mesh use exact four-map crossing coverage plus complete owner/control/top-port coverage. Each category below counts directed destination deliveries. Clock, reset-only and data/functional-control categories are mutually exclusive and add to the total. These original-source counts include the top-port policy of the proof, so they differ from optimization transfers.

Design	FPGAs	Crossing nets	Destinations	Clock	Reset only	Data/control
OpenTitan	1	0	0	0	0	0
OpenTitan	2	1,989	1,989	1	3	1,985
OpenTitan	3	3,187	3,277	2	14	3,261
OpenTitan	4	3,352	3,676	3	25	3,648
NVDLA	1	0	0	0	0	0
NVDLA	2	3,239	3,239	1	2	3,236
NVDLA	3	6,230	6,234	2	4	6,228
NVDLA	4	8,313	8,320	3	5	8,312
Mesh	1	0	0	0	0	0

Design	FPGAs	Crossing nets	Destinations	Clock	Reset only	Data/control
Mesh	2	3,715	3,715	1	7	3,707
Mesh	3	6,504	6,549	2	10	6,537
Mesh	4	7,178	7,268	3	8	7,257
AES	1	0	0	0	0	0
AES	2	8,486	8,486	1	40	8,445
AES	3	13,937	14,227	2	90	14,135
AES	4	16,326	16,915	3	109	16,803

B Technical definitions for developers

The human-facing tables use full names. Raw developer files retain their stable field names for compatibility. This section gives an exact translation.

Let a modeled path visit primitives $v_0, \ldots, v_m$, and let $a(v)$ be a primitive's FPGA owner. Its boundary-crossing count is

$$\text{crossings}(p) = \sum_{j=1}^{m} \mathbf{1}[a(v_{j-1}) \neq a(v_j)].$$

The raw field `C` is the largest such count over modeled paths. If $h(x, y)$ is the shortest declared physical-link count between selected FPGAs, then the raw field `H` is

$$\max_{p} \sum_{\substack{j=1 \\ a(v_{j-1}) \neq a(v_j)}}^{m} h(a(v_{j-1}), a(v_j)).$$

It is the worst minimum-link hop count, not a chosen transport route. J4's selected pairs have one-hop adjacency, so the two maxima coincide in these tables.

For each capture or terminal endpoint, take its largest incoming crossing count minus one, clamped at zero. Sum that over endpoints to obtain the raw `E` field, called *extra crossings at endpoints* in this report. The raw `V` field counts the combinational transit primitives described earlier. The raw `T` field counts directed structural net-destination transfers in the optimizer's narrower scope.

The final path audit uses dynamic programming over all supported acyclic dependencies. Local move scoring uses a bounded witness set: at most 4,096 complete endpoint-maximal witnesses and a one-million primitive-ID aggregate default budget. That limit bounds refinement work; it is not the coverage limit of the final structural audit.

B.1 Primitive behavior

The selected model is `functional_synchronous_controls_v2_UNISIM_2025_1`. A sequential primitive captures incoming dependencies and launches an independent outgoing path. Functional synchronous reset/set and enabled internal stages participate according to their parameters. Asynchronous reset/preset or disabled stages do not become synchronous captures just because a pin name looks like a control.

Memory write/data/control captures are distinguished from asynchronous LUTRAM/SRL read-address dependencies. BRAM, URAM and DSP internal stage settings affect the model. Missing settings use

the pinned UNISIM 2025.1 defaults. Invalid or unsupported settings cannot be assigned invented safe behavior. Unknown primitives and combinational cycles limit the claimed completeness. This is a conservative supported graph, not a full Boolean or pin-arc timing proof.

B.2 Source timing weights

For an exported path with source slack s_p and source requirement R_p, the criticality is

$$q_p = \max(0, 1 - s_p/R_p)^2.$$

Normalize by the largest positive criticality in the exported sample, or by one when there is none. Eligible initial hyperedges then receive one plus a bounded rounded timing weight based on the largest matched normalized criticality. Matching uses ordered owner pairs and can conservatively affect parallel hyperedges; it is not exact pin-arc attribution.

The sampled path score combines criticality, topology hops, additional crossings and returns to an earlier FPGA. Without an explicit delay model, the retained-candidate ranking prioritizes worst minimum-link hops, worst logical crossings, sampled timing cost, endpoint excess and transit cells, followed by the remaining acceptance tiers. With a supplied extra delay per logical crossing, modeled sample slack subtracts that delay times the logical crossing count. This option does not discover omitted paths or measure real transport delay.

B.3 Capacity and balance arithmetic

For each target, the effective Native LUT cap is the minimum of its selected utilization fraction times its actual part capacity, any uniform explicit cap, any per-device explicit cap, and any topology cap. Unspecified terms are omitted, and the utilization fraction cannot exceed 75%.

For equal parts, reported imbalance is

$$\frac{\text{largest Native LUT count}}{\text{mean Native LUT count}} - 1.$$

For unlike parts, selection uses the analogous imbalance of capacity-normalized usage, while retaining raw imbalance separately. An experimental lower cap does not redefine a device's true capacity.

The preferred balance target, allowed search envelope and hard capacity are separate concepts. Cut/topology growth allowances remain anchored to the original reference through refinement; they do not grow on every accepted pass. Ranked acceptance can permit a lower-priority regression when a higher-priority measure improves. Componentwise acceptance forbids each listed regression. Hard capacity, useful occupancy and original budget checks remain separate gates.

In the reporting revision, the automatic estimated scalar search envelope is the larger of the preferred balance and $\sum_i L_i / \min_i L_i - 1$, using physical selected-part capacities. For equal devices this becomes the number of selected FPGAs minus one. The default raw logical-LUT guard is the larger of 0.5 and that equal-device bound. Explicit search/raw guard flags still tighten the search; Native physical caps remain hard. These new defaults broaden exploration, but do not by themselves establish a measured quality improvement over the historical runs.

C Evidence files and what they establish

The repository keeps the report source, its rendered PDF, figure sources and small evidence extracts together. The central extracts are:

`evidence/validation_evidence.json` The original selected-case metrics, comparison traces and report-identity index, extracted from 343 bounded small reports. It does not contain the large EDIF/DCP payloads or replace the original Native receipts.

`evidence/mapping_contract_validation.json` The 20 October exact-hierarchy audit, corrected mapping identities and ownership-equivalence results for all sixteen cases.

`evidence/release_validation.json` Historical clean-clone and unpacked-runtime test receipts. These identify their own tested code versions; they are not silently reassigned to later revisions.

`evidence/presentation_evidence.json` Plain-English numeric data for the paper figures, including checked-in selected-case results and the preserved OpenTitan attempt history. Original input SHA-256 values and missing-measurement markers are retained. Its generator reads existing evidence only.

OpenTitan search history The final summary is `FINAL_EXPERIMENT_SUMMARY.json`; the per-attempt table is `iteration_progress.csv`. Both belong to the preserved `work/opentitan_j4_70pct_20261015` experiment. They distinguish resource-measured candidates, rejected proposals and independently checked selected maps.

The evidence uses a `workspace/` alias for the common parent of RapidWright, LSUT_2, topology, work and results directories. Relocating an alias does not change what the original hash identifies. A successful extraction of small reports is a consistency check, not a new solver or Vivado execution.

C.1 The large mesh boundary-extraction repair

An earlier mesh clock comparison missed 18,432 logical RAM WCLK endpoints: 6,144 RAM32M and 12,288 RAM32M16. Vivado's implementation-leaf traversal is not the same as the original logical EDIF macro boundary. Adding the missing source tokens as a diagnostic reproduced the expected Java fingerprint, but that reconstruction alone was not accepted as Native proof.

The final Native producer queried all electrical segments with ordinary `get_pins -of_objects`, without `-leaf`, in bounded 512-segment chunks. It captured pin name, reference-pin name and direction before collection stringification, checked aligned columns and rejected conflicting duplicate metadata. A one-time top-port index retained original port identities and alias membership. The final unchanged comparator then checked the complete selected-map scope.

The completed mesh Native phase took 4,715.16 seconds at about 43.22 GB peak RSS; the comparison took 240.32 seconds and exited zero. These are workstation validation costs, not partitioner throughput claims. Failed or stopped probes remain historical failures; they do not become successful proof because a later run finished.

C.2 Historical clean-clone and runtime checks

The pre-contract release at `f9f2cada5b8151df08414188e6bdfc554f6ce695` was built from a fresh local clone without inherited classes, jars, data or checkout build output. It reused an allowed Gradle dependency cache. The focused Java execution passed 120 tests with no skips; Python completed 172 tests, with 170 passing and two optional recorded-Native fixtures skipped. These are version-specific counts, not numbers to add to later repeated tests.

That release's unpacked runtime passed tool preflight and actual one-, two-, three- and default-four-FPGA runs on a small source. Its Native LUT counts were 304; 185/119; 117/102/85; and 70/68/83/83. A two-FPGA request with a one-LUT hard cap was rejected after actual counts exceeded it, with no accepted `mapping.txt`. Those values describe the historical executable. The 20 October four-FPGA fixture result was 85/83/68/68 under its separate revision and must not be substituted into the old receipt.

The 20 October connection regression also covers a wholly internal two-alias net below a portless coarse frontier. It requires one canonical net and two endpoint rows without changing historical cut counters. A real two-FPGA fixture retained every connection row and byte-identical crossing/category tables after the memory-use improvement.

C.3 Reproducing a result rather than just rerunning a command

Retain the original source checkpoint/EDIF, logical inventory, timing sample and metadata, topology and part assignment, Mt-KaHyPar version, Vivado version, primitive model, caps, effort, seeds, feedback state, budget reference and selected iteration. Their identities are recorded in `run_provenance.json`, `STATUS.json` and comparison receipts. A short command with current defaults is not an exact reproduction of every historical experiment.

The bounded snapshot can be regenerated from the preserved workspace reports:

```
python doc/partition/evidence/build_snapshot.py --workspace <common-parent>
```

It reads small completed reports. It does not regenerate missing measurements, rerun STA, synthesize a design or verify a missing source payload. Deeper verification requires the original files and installed tools/licenses.

C.4 Key original identities

Hashes below locate original bytes. They provide traceability, not a substitute for reading the underlying evidence. The approved large-design engine manifest is

`b12fe61853478f92424932bcbbe7b0c82635c8f158de5a888bac8a2766cc48b2.`

The synchronous-control model authority manifest is

`60bc13f5988f730bf9dcd5936ff37bd639e3cb06700e5813c61ed469a25ad3b5.`

The unchanged original J4 topology JSON is

`6f393e7eee76cbf20cd96e46f52c8c0a34be2365c7222aa6cf7abfe87c7e1b83.`

The original selected J4 summary is

`1c222fdb23eaae252b04ec7f843ba208eae9f352ef3f417d5b6a50779dc01d92.`

The completed mesh Native receipt and exact parity report are respectively

`52f3e4f8f0fdd334424235a8c8261b30c03e32e90f3709b3dc0fd8f885a1c96e`

`593b40915c6dbae5997e0412f79b5d9231a1ebc45a1229f91a9ae79f178bd785.`

The historical clean-clone and unpacked-runtime receipts are respectively

```
24a26d1bbb382483cd2589d21f9aa5a943ff70b9f81cb62553b27757638a8e04
```

```
2d35ab553d0a14ad7e87a570cfddef348699e14859438feb8c337165509ecdc1.
```

The old tested runtime ZIP and its non-documentation code manifest are respectively

```
9523d4ca1d3db88ec8d9e78216c68eaf6912ba568b787102dbbbadef0cdf0810
```

```
499dad8d823e4ed852f33fe72604648840a01380c57cb95e1ea6d49525c3d5d2.
```

Those runtime identities describe the pre-contract executable. The current branch is `Partitioner_Extreme_Overhaul`. Later mapping-contract and reporting revisions have their own tests and Git history.

Check the source revision with `git rev-parse HEAD`. An external checksum sidecar identifies a final release archive without embedding a circular self-hash inside its own PDF.

D Historical mapping identities

These hashes identify the original measured ownership files, including the four move representations later repaired. They are not a declaration that those four old files are legal consumer inputs. The successor identities and equivalence receipts are in `evidence/mapping_contract_validation.json`. Use the corrected package for move files.

Design / FPGA count	Original compact mapping SHA-256
OpenTitan / 1	`c2ea38c33d11023cf078f580cb53bbd84e0a0c93f45a94a1a374321bbb5dfc16`
OpenTitan / 2	`e936356c04c73c0d99bd11538e731007aa2ba28da84ccd12c47d49a608cee04a`
OpenTitan / 3	`fa6fa3113fa99a3905c8ae3c6ee9ac1c427d3da5a4afaf52a6a73a479969ea92`
OpenTitan / 4	`574a96b4079dce7d6c9ebb9a7908d3d3cfb835fba616a6e27c4320a807d6def9`
NVDLA / 1	`1c4a2ad3b2f82091a816586fd118177b0a06a3777ea0855fcfcd0be354041102`
NVDLA / 2	`b9f20eccfbc28683314417a1d32fa0ff40928480a7553d2214cec28c4ad99e11`
NVDLA / 3	`8466751633f900eb0f46491878d1f0294afa5e9eae002b9b8f2ac84840d9a5c1`
NVDLA / 4	`51de3c6b16c4f06c102b42c7a59262ec7f1e69ed56504ee9b13bc352905e0383`
Mesh / 1	`962a2da742da514bc29fb85f8cac59cf2ab1d571b84e9219d95b0634740cb508`
Mesh / 2	`6712b6b70b31e1726d633499171b94f048e0f4ed906229bb556bcce011b95470`
Mesh / 3	`93fce6c9f77e7e39199a8a74b40d98132d9299545ac29f6be7c6149eda4dcb45`
Mesh / 4	`e200219ac0ef2c9ad7eb4f8712ffca177a090d7e7e168d1e162eec9d1476c37f`
AES / 1	`b7c6ffa571a33ca19d6d42a6643983836957089d2af3b303ed31f9ef257fbd23`
AES / 2	`cf236d659341e88c84cff660e58af2c50eed10fdb6e8834f8333560b4486c877`
AES / 3	`206ec1934041f63b0d98c27c1ba9357ba4d7c76832c288b234853746a5c501b6`
AES / 4	`5557dfb34e40a282e7a84127f4e24f416d8f3bf6de4b938e4d078d67810aeac2`

References

[1] OpenROAD Project, "Partition Manager: Timing Aware Algorithm," official documentation, accessed 15 October 2026. `https://openroad.readthedocs.io/en/latest/main/src/par/README.html#timing-aware-algorithm`.

[2] I. Bustany, G. Gasparyan, A. B. Kahng, I. Koutis, B. Pramanik and Z. Wang, "An Open-Source Constraints-Driven General Partitioning Multi-Tool for VLSI Physical Design," ICCAD, 2023, doi:10.1109/ICCAD57390.2023.10323975. Author paper: `https://vlsicad.ucsd.edu/Publications/Conferences/401/c401.pdf`.

[3] Mt-KaHyPar maintainers, official README at tag `v1.6.2`, accessed 15 October 2026. `https://raw.githubusercontent.com/kahypar/mt-kahypar/v1.6.2/README.md`.

[4] Mt-KaHyPar maintainers, official file-format specification at tag `v1.6.2`, accessed 15 October 2026. `https://raw.githubusercontent.com/kahypar/mt-kahypar/v1.6.2/mt-kahypar/io/docs/FileFormats.md`.

[5] L. Gottesbüren, T. Heuer, N. Maas, P. Sanders and S. Schlag, "Scalable High-Quality Hypergraph Partitioning," 2023. `https://arxiv.org/abs/2303.17679`.

[6] T. Heuer, "A Direct k-Way Hypergraph Partitioning Algorithm for Optimizing the Steiner Tree Metric," ALENEX, 2024; author version 2023. `https://arxiv.org/abs/2309.16694`.

[7] T.-W. Lin, W.-C. Tai, Y.-C. Lin and I. H.-R. Jiang, "Routing Topology and Time-Division Multiplexing Co-Optimization for Multi-FPGA Systems," 2020. Author paper: `https://waynelin567.github.io/files/FPGA_TDM.pdf`.

[8] D. Zheng and E. F. Y. Young, "An Integrated Circuit Partitioning and TDM Assignment Optimization Framework for Multi-FPGA Systems," ASP-DAC, 2023. Publisher abstract: `https://ieeexplore.ieee.org/document/10044749`.

[9] S. Tong, H. Li, J. Xu, C. Pei, W. Yu, S. Liu and J. Shen, "EasyPart: An Effective and Comprehensive Hypergraph Partitioner for FPGA-based Emulation," ICCAD, 2024. Author paper: `https://numbda.cs.tsinghua.edu.cn/papers/iccad24.pdf`.

[10] Synopsys, "Prototyping Systems / HAPS ProtoCompiler SW," official product description, accessed 15 October 2026. `https://www.synopsys.com/verification/emulation-prototyping/prototyping.html`.